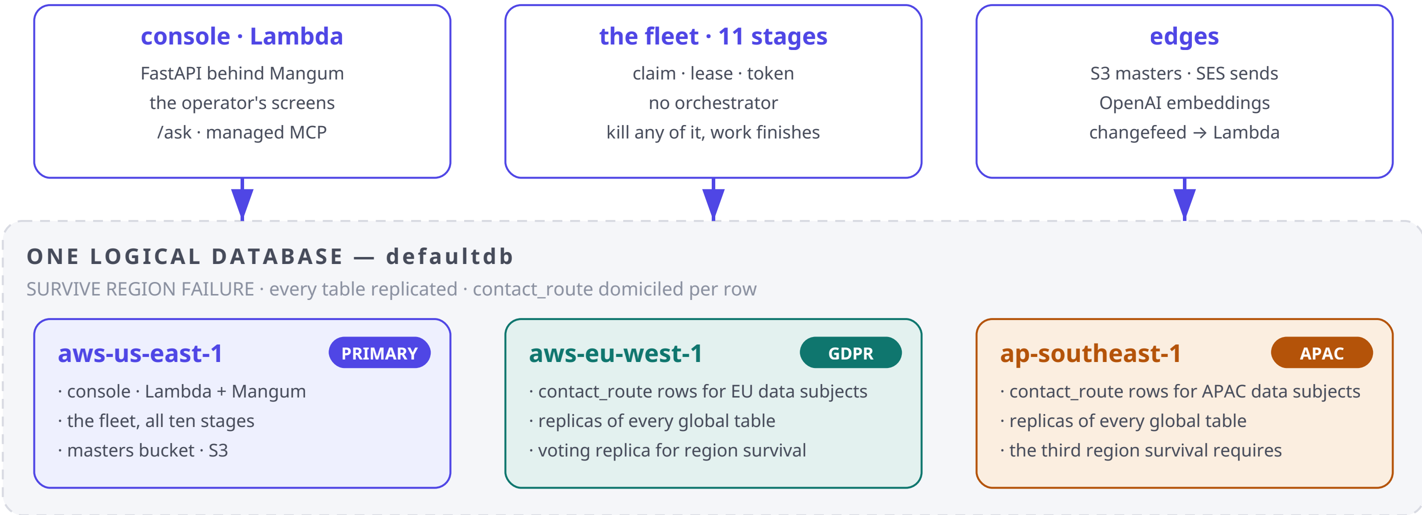
AS BUILT · 2026-08-13 · WITH THE MULTI-REGION TOPOLOGY DEPLOYED

Respect the Funk — the system, as it actually works

Every number on these pages was read off respect-the-funk (CockroachDB CCL v26.2.5) on 2026-08-13. The three-region topology is drawn as deployed because that is what this document assumes; page 6 is the ledger of what is running right now and what is one command away. A diagram of a system that does not exist yet is a prospectus, and the honest way to publish one is to print the gap beside it.



ONE SUBSTRATE, FIVE JOBS

index	scheduler	outbox	event bus	audit log
R1 + R2 vector search	lease + SKIP LOCKED	at-most-once sends	changefeed	AS OF SYSTEM TIME

Not Postgres plus a vector store plus Redis plus a queue plus an audit pipeline.
One system. Five jobs. Zero nodes at rest.

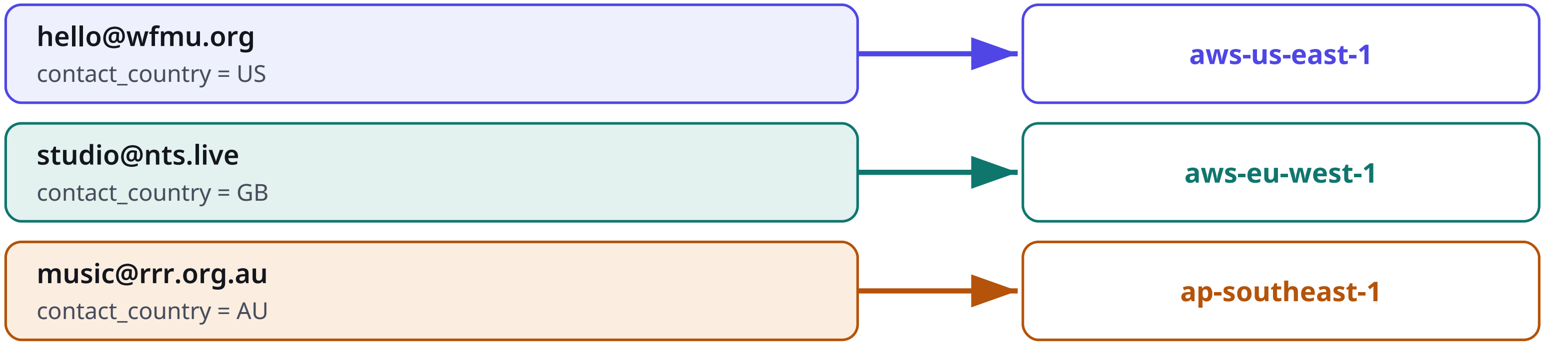
14,170	14,136	22,057	45,125	54,183	$0.12
COUNTERPARTIES	EMBEDDED	EVIDENCE CHUNKS	FACTS	AGENT RUNS	LIFETIME SPEND

One table, three continents, enforced by the database

The agents email real people, so the system holds real personal data about them, and European law says where that data may live. `contact_route` is one logical table whose rows are domiciled individually — this is the capability no single Postgres has.

contact_route — ONE TABLE

REGIONAL BY ROW AS residency_region · a STORED computed column derived from contact_country



WHAT THE DATABASE ENFORCES, NOT THE APPLICATION

Same table. Same query. The row for an EU data subject has its replicas physically in Ireland.
No ELSE branch in the region mapping — an unmapped country fails NOT NULL rather than being domiciled somewhere plausible. CN and RU are unmapped deliberately: Singapore does not satisfy PIPL and nothing here satisfies 152-FZ.
In Postgres this is three databases, an application-level router, and a synchronisation problem you own forever. Here it is a table property.

A REGION CANNOT BE REMOVED ONCE ADDED — on Basic or Standard.
So the runbook provisions a throwaway cluster — converting the live one would be permanent, and its teardown would mean a backup, a new cluster, a restore, and repointing every URL.

Filtered vector search, and what each index is actually worth

Two retrievals do real work and are measured here. A third has one row in it. A fourth was deleted rather than backfilled, because it indexed a column nothing wrote for a query nobody ran — and an index you removed is more honest than one that indexes nothing.

R1 — who should we pitch?

party@party_shortlist

`tenant_id · embedding_model · party_class · contact_state`

`→ vector search with prefix spans`

`14,136 vectors · 23,891 reads`

R2 — what do we already know?

party_chunk@chunk_semantic

`tenant_id · model`

`→ vector search with prefix spans`

`22,057 chunks · 45,238 reads`

THE FILTERS ARE INSIDE THE INDEX

```
• vector search
table: party@party_shortlist
prefix spans: [/'1f9e6dd3…'/'openai:text-embedding-3-small'/'counterparty'/'contactable' - …]
```

The search runs in the filtered subspace — not a scan with a WHERE clause bolted on after. And that prefix starts with tenant_id, the same column a static test proves every statement in the codebase carries. One column: security boundary and partition key at once.

ALL FOUR VECTOR INDEXES, HONESTLY

party_shortlist	14,136 vectors	live	R1. Four prefix columns — the ANN search runs in the filtered subspace.
chunk_semantic	22,057 chunks	live	R2 over evidence. Second most-read index in the whole cluster.
lesson_semantic	1 row	thin	The memory loop. Works, and holds one lesson — one thread has closed.
fact_semantic	—	dropped	Indexed a column nothing wrote, for a query nobody ran. Removed.

The database is the runtime

Agents never call each other. A row changing is the scheduling primitive, and the fence that decides who may act on it is a capability token only the database mints.

a row changes — thread · outbox · message → **CHANGEFEED** — initial_scan = 'no' → **Lambda webhook** — wakes the next stage

No broker. No queue. No scheduler. Waking is not claiming — two workers woken by the same event still produce exactly one winner.

AND ONLY ONE OF THEM WINS

worker A	`owner=ingest-cli · token 2bb498ed…`	**REFUSED — lease lost**
worker B	`owner=ingest-cli · token 6f72beda…`	**OK**

Same name. Same lead. The name cannot tell them apart; the token can — and only the database mints it.
Serializable by default, FOR UPDATE SKIP LOCKED, and a fence that fails closed.

THE ELEVEN STAGES IN agents.REGISTRY

`embed_document`	`map_source`	`find_counterparties`	`profile_party`
`embed_party`	`distil_lesson`	`analyse_recording`	`index_stations`
`index_streams`	`enrich_genre`	`index_podcasts`	

Every one claims its own work. Agents never call each other — 54,183 runs recorded, 36,495 leads in the frontier.

What did it believe, when it acted?

The hardest question any agentic memory faces is not what the agent knows — every RAG system answers that. It is why it decided that, then.

```
thread.decided_at_hlc = 1786644836824776765.0000000000
a hybrid logical clock reading — not a timestamp, and the difference is the whole design
```

AS OF SYSTEM TIME with a wall clock resolves to the most recent version at or before that instant, and several writes share a millisecond. Landing on the wrong side of one reconstructs a ranking nobody ever saw — a rounding error in a chart, a fabrication in the record that justifies emailing a stranger, and indistinguishable from the truth either way.

THREE OUTCOMES, KEPT APART ON PURPOSE

`a ranking`
the shortlist as it stood, plus whether the replay still agrees with what was recorded

`NotJustified`
nobody ranked this — a human opened the thread, and an absent reason must look absent

`HistoryExpired`
a reason was recorded and the GC window has passed. Never falls back to today's ranking

Every agentic system can tell you what it knows. This one tells you what it believed, when it acted.

Four extra words of SQL. No audit table, no event log, no versioned copy of anything — and no way to have chosen in advance what to record, because AS OF SYSTEM TIME replays state nobody chose to record, including the vector ranking itself.

Running, built, or blocked

The page that makes the rest of this document honest. Everything above is drawn as though deployed; this is what is true on 2026-08-13, and how each line was checked.

Filtered vector search (R1) — running
EXPLAIN shows a vector search node with four prefix spans. 23,891 reads.

Evidence retrieval (R2) — running
22,057 chunks over 14,018 parties. 45,238 reads.

Time travel on the production path — running
shortlist_as_of takes an absolute HLC; the scrubber returns a different first result at -1h than at now.

Decision provenance — running · no data
Schema, writer, replay and panel all land. 0 threads carry one — 1 thread exists and it predates the change.

Serializable claiming with a lease token — running
lease_race_demo Act 2: two workers, one name, only the token tells them apart.

Tenant isolation proved statically — running
test_tenant_scoping resolves every statement reaching execute() and fails the build on any that is unscoped.

Suite isolated from production — running
A marker table the cluster must carry. Production has none, so the suite refuses.

Managed MCP server in the console — running
Six pinned SQL literals; the model classifies onto them and never writes SQL.

Changefeed wakes the agents — built, not created
Statement, consumer, Lambda sink and --verify all exist. SHOW CHANGEFEED JOBS returns 0 — it draws RUs until cancelled, so creating it stays a human step.

REGIONAL BY ROW residency — built, not provisioned
Terraform, migration 024 and the runbook exist. SHOW REGIONS returns one row. About $0.65 for a three-hour window on a throwaway cluster.

Bedrock embeddings — blocked at the account
On-demand quota 0 and non-adjustable. Batch is not authorised pending a support case. Embeddings are OpenAI; AWS is Lambda and S3, which are real.

Podcast corpus — built, no credential
Adapter, migration, ingest stage and worker are tested. Needs a free API key.

Three of these are one action away and none has been taken: paste the changefeed statement into `cockroach sql`; run `docs/runbooks/multiregion.md` (~$0.65, three hours, throwaway cluster); request a free Podcast Index key. The fourth — Bedrock — is blocked at AWS and needs a support case.